Exact SAT Solving and Constraint Programming for the N-Queens Problem

Pham Vinh Tri[a]

[a] *Faculty of Information Technology, VNU University of Engineering and Technology, 144 Xuan Thuy, Cau Giay, Hanoi, 100000, Vietnam*

ABSTRACT

The N-Queens problem is a classic constraint satisfaction benchmark requiring the placement of N non-attacking queens on an $N \times N$ chessboard. We study it as a controlled testbed for SAT encodings of At-Most-One (AMO) constraints. Five AMO encodings—Binomial (Pairwise), Sequential Counter, Binary, Commander and Product—are formulated, implemented and verified: all five are shown mechanically to be correct and to enforce arc consistency through unit propagation, so they do not differ in propagation strength. They are benchmarked with Glucose3 for N from 8 to 200 and compared with Constraint Programming (OR-Tools CP-SAT, IBM CPLEX CP Optimizer) and Integer Linear Programming (IBM CPLEX, Gurobi). The encoding with the largest CNF, Binomial, is the fastest SAT encoding on the original order up to $N = 100$ and never times out on any of the ten orders tested at $N = 30, 50, 100$, so CNF size does not predict solving time. A permutation study over ten equivalent variable and clause orders per configuration shows, however, that single-order measurements greatly exaggerate the differences between encodings: at $N = 30$ the slowdowns of Product, Binary and Commander shrink by factors of 1083, 515 and 32, respectively. Genuine differences emerge only at larger sizes, where Sequential, Binary and Product fail on most orders at $N = 100$ while Commander remains as fast as Binomial. A five-backend comparison (Glucose3/4, CaDiCaL, MapleSAT, CryptoMiniSAT5) shows that the best backend depends on the encoding. A first-feasible-solution experiment confirms that stopping criteria do not explain the MIP results: Gurobi finds a solution at the root node for $N \leq 30$ and after a single node at $N = 40$. Finally, a noise analysis and a list of threats to validity delimit which orderings the data support.

1 Introduction

The N-Queens problem is a classical benchmark for combinatorial search and constraint reasoning [1]. The objective is to place N queens on an $N \times N$ chessboard such that no two queens attack each other under standard chess rules (i.e., sharing neither row, column, nor diagonal). The number of candidate placements grows factorially with N, so naive depth-first backtracking quickly becomes impractical.

It is important to state the scope of this study precisely. Finding *one* solution of N-Queens is not computationally hard: explicit constructions place N non-attacking queens in $O(N)$ time for every $N \geq 4$ [1]. The problem is used here not because it requires SAT, but because its constraint structure—a set of overlapping At-Most-One (AMO) constraints—is simple, fully understood, and scalable, which makes it a controlled testbed for comparing CNF encodings and solver paradigms.

This study presents a SAT-based methodology for solving the N-Queens problem exactly and examines how the choice of AMO encoding affects Conflict-Driven Clause Learning (CDCL) solvers. Beyond reporting runtimes, we check two methodological pitfalls that are easy to overlook: (i) whether differences in propagation strength actually exist between the encodings, which we verify mechanically, and (ii) whether runtime differences survive a change of variable and clause order, to which CDCL solvers are known to be sensitive. The remainder of this report is organized as follows: Section 2 reviews related work. Section 3 formalizes the problem. Section 4 establishes five SAT encodings for AMO constraints along with symmetry breaking. Section 5 presents the Constraint Programming (CP) formulation. Section 6 provides experimental benchmarks, an ordering-sensitivity study, scalability evaluations, cross-solver comparisons, fairness analyses, and threats to validity. Section 7 concludes the paper.

2 Related Work

Transforming cardinality constraints into Conjunctive Normal Form (CNF) is a foundational and actively developed area in propositional satisfiability. Warners [2] proposed linear-time transformations for linear inequalities using auxiliary variables. Sinz [3] subsequently introduced the Sequential Counter encoding, achieving linear $O(n)$ clause complexity and establishing a standard building block in modern SAT applications. Gent [4] showed how a CNF translation can let unit propagation enforce arc consistency, and Bailleux and Boufkhad [5] applied this criterion to cardinality constraints, making propagation strength a standard yardstick alongside encoding size. For the specific At-Most-One (AMO) constraint, Klieber and Kwon [6] proposed the Commander encoding, while Chen [7] designed the Product encoding based on 2D coordinate decomposition.

In terms of solver architectures, Conflict-Driven Clause Learning (CDCL) equipped with Variable State Independent Decaying Sum (VSIDS) decision heuristics [9] constitutes the core engine of Glucose [10], the primary SAT solver evaluated in this study. CDCL solvers are also known to be sensitive to seemingly irrelevant properties of the input, such as the order of variables and clauses: runtimes of complete search methods on the same instance can vary by orders of magnitude under randomisation [8], so single-run comparisons can be misleading. In parallel, Constraint Programming (CP) [11] preserves high-level integer constraint structures; the `AllDifferent` constraint admits a generalized-arc-consistent filtering algorithm based on bipartite matching [12], although practical solvers choose their propagation level per constraint. Integer Linear Programming (ILP) / Mixed Integer Programming (MIP) [13] formulates the constraints as linear inequalities solved via presolve, primal heuristics and branch-and-cut. Our study evaluates all three paradigms (SAT, CP, ILP) side by side on a uniform benchmark.

3 Problem Formulation

The N-Queens problem can be formally modeled as a Constraint Satisfaction Problem (CSP). Given an $N \times N$ chessboard, let $x_{i,j} \in \{0, 1\}$ denote binary decision variables indicating whether a queen is placed at row i and column j, where $i, j \in \{1, \dots, N\}$. The constraint system is defined by:

$$\sum_{j=1}^{N} x_{i,j} = 1 \quad \forall i \in \{1, \dots, N\} \tag{1}$$

$$\sum_{i=1}^{N} x_{i,j} \leq 1 \quad \forall j \in \{1, \dots, N\} \tag{2}$$

$$\sum_{i-j=k} x_{i,j} \leq 1 \quad \forall k \in \{-(N-1), \dots, N-1\} \tag{3}$$

$$\sum_{i+j=k} x_{i,j} \leq 1 \quad \forall k \in \{2, \dots, 2N\} \tag{4}$$

Equation (1) enforces the Exactly-One constraint for every row. Equations (2), (3), and (4) enforce At-Most-One (AMO) constraints across columns and both sets of diagonals. Because an Exactly-One constraint naturally decomposes into an At-Least-One (ALO) clause and an AMO constraint, the core mathematical challenge lies in optimizing the CNF representation of AMO constraints. Columns need only AMO: N queens in N distinct columns already fill every column. (The MIP models state the column constraints as equalities, which is equivalent.) In the SAT encodings the cell (i, j) is the propositional variable with index $(i-1)N+j$, and every AMO is applied only to diagonals with at least two cells.

4 SAT Encodings for At-Most-One Constraints

To leverage modern CDCL SAT solvers, the constraint system must be transformed into Conjunctive Normal Form (CNF). Given a set of boolean variables $X = \{x_1, \dots, x_n\}$, the AMO constraint enforces $\sum_{i=1}^{n} x_i \leq 1$. Below is the mathematical formulation of the five evaluated encodings.

4.1 Binomial (Pairwise) Encoding

The most intuitive formulation enforces mutual exclusion across all pairs of variables. For every pair $i \neq j$, both variables cannot simultaneously evaluate to True:

$$\mathrm{AMO}_{\mathrm{Binomial}}(X) = \bigwedge_{1 \leq i < j \leq n} (\neg x_i \vee \neg x_j) \tag{5}$$

This direct encoding introduces zero auxiliary variables. However, it incurs quadratic space complexity with $\binom{n}{2} = O(n^2)$ binary clauses.

4.2 Sequential Counter Encoding

To reduce clause volume, Sinz [3] introduced $n-1$ auxiliary variables $s_1, \ldots, s_{n-1}$ acting as an accumulator register. The clauses force s_i to be True whenever some x_k with $k \leq i$ is True (the converse is not enforced), and forbid x_i when s_{i-1} is already True:

$$(\neg x_1 \vee s_1) \tag{6}$$

$$\bigwedge_{i=2}^{n-1} ((\neg x_i \vee s_i) \wedge (\neg s_{i-1} \vee s_i) \wedge (\neg x_i \vee \neg s_{i-1})) \tag{7}$$

$$(\neg x_n \vee \neg s_{n-1}) \tag{8}$$

This encoding achieves a linear clause count of $3n - 4 = O(n)$, at the cost of introducing $n - 1 = O(n)$ auxiliary variables.

4.3 Binary, Commander, and Product Encodings

Binary Encoding: Let $m = \lceil \log_2 n \rceil$ and introduce auxiliaries $b_1, \ldots, b_m$. Writing $\beta_t(k) \in \{0, 1\}$ for the t-th bit (most significant first) of the m-bit binary representation of $k-1$, each x_k forces the bit pattern of its own index:

$$\mathrm{AMO}_{\mathrm{Binary}}(X) = \bigwedge_{k=1}^{n} \bigwedge_{t=1}^{m} \left(\neg x_k \vee b_t^{\beta_t(k)}\right), \qquad b^1 = b,\ b^0 = \neg b. \tag{9}$$

Two true variables would demand two different bit patterns, which is impossible. This yields exactly $n\lceil \log_2 n \rceil$ binary clauses and $\lceil \log_2 n \rceil$ auxiliary variables.

Commander Encoding: Partitions X into consecutive groups $G_1, \ldots, G_g$ of size $s = \lceil \sqrt{n} \rceil$ (the last group may be smaller), so $g = \lceil n/s \rceil$ [6]. Each group has a commander c_h:

$$\mathrm{AMO}_{\mathrm{Cmd}}(X) = \mathrm{AMO}_{\mathrm{Binomial}}(c_1, \ldots, c_g) \wedge \bigwedge_{h=1}^{g} \left(\left(\neg c_h \vee \bigvee\nolimits_{x \in G_h} x\right) \wedge \bigwedge\nolimits_{x \in G_h} (\neg x \vee c_h) \wedge \mathrm{AMO}_{\mathrm{Binomial}}(G_h) \right), \tag{10}$$

so that $c_h \Leftrightarrow \bigvee_{x \in G_h} x$. Our implementation is the one-level variant (Klieber and Kwon apply the scheme recursively); it uses $g = O(\sqrt{n})$ auxiliaries and $\binom{g}{2} + \sum_h \left(1 + |G_h| + \binom{|G_h|}{2}\right) = O(n\sqrt{n})$ clauses. For $n \leq 4$ it falls back to Binomial.

Product Encoding: Arranges the variables into a 2D matrix of dimensions $p \times q$ with $p = \lceil \sqrt{n} \rceil$, $q = \lceil n/p \rceil$ [7]; variable x_k occupies row $r(k) = \lfloor (k-1)/q \rfloor + 1$ and column $c(k) = ((k-1) \bmod q) + 1$. With row auxiliaries $R_1, \ldots, R_p$ and column auxiliaries $C_1, \ldots, C_q$:

$$\mathrm{AMO}_{\mathrm{Product}}(X) = \mathrm{AMO}_{\mathrm{Binomial}}(R) \wedge \mathrm{AMO}_{\mathrm{Binomial}}(C) \wedge \bigwedge_{k=1}^{n} \left(\neg x_k \vee R_{r(k)}\right) \wedge \left(\neg x_k \vee C_{c(k)}\right). \tag{11}$$

These forward implications are exactly the clauses of Chen's encoding; we encode the two inner AMO constraints pairwise instead of recursively, which yields $2n + \binom{p}{2} + \binom{q}{2} \approx 3n$ clauses and $p + q = O(\sqrt{n})$ auxiliary variables. Because only forward implications are used, an auxiliary may be True while no variable in its row or column is True. This does *not* weaken propagation on the original variables: if x_k is True, unit

propagation sets $R_{r(k)}$ and $C_{c(k)}$, falsifies every other row and column auxiliary, and thereby falsifies every other x_j (two variables sharing a row differ in their column). For $n \leq 4$ the encoding falls back to Binomial.

> **Technical Note: PySAT Cardinality Encoding Semantics**
>
> **Note on PySAT encoding labels.** In addition to the five handwritten implementations in `encoder.py`, our codebase includes `pysat_encoder.py`, which interfaces directly with PySAT's cardinality algorithms (`pysat.card.CardEnc`). It should be noted that certain PySAT encoding identifiers do not map one-to-one to the algorithms named above: specifically, `CardEnc` types `cardnetwrk` (Cardinality Networks) and `mtotalizer` (Modulo Totalizer) are generalized sorting/counter networks with topological structures distinct from our handwritten Commander and Product encodings. Therefore, the PySAT module serves primarily as an independent verification baseline for logical correctness: the unit tests enumerate every solution for small N under each handwritten and each PySAT encoding and check that the solution sets coincide with a brute-force enumeration. All quantitative comparisons and structural evaluations in this report originate strictly from the five native implementations in `encoder.py`.

4.4 Symmetry Breaking

The solution set of N-Queens is closed under reflection across the vertical axis of the board: if a placement is valid, its mirror image is also valid. We eliminate roughly half of the search space without loss of satisfiability by restricting the queen of the first row to the left half of the board (indices are 1-based as in Section 3), which adds $\lfloor N/2 \rfloor$ unit clauses:

$$\neg x_{1,j} \quad \forall j \in \{\lceil N/2 \rceil + 1, \dots, N\} \tag{12}$$

For odd N the middle column remains allowed, which keeps the constraint sound. These unit clauses are injected identically across all five SAT encodings, so all SAT configurations search the same reduced space. The CP and MIP models (Section 5 and the MIP counterpart of Eqs. (1)–(4)) do *not* include this constraint; since it only removes solutions that are mirror images of others, its effect on time-to-first-solution is small, but the asymmetry is listed among the threats to validity (Section 6.10).

5 Constraint Programming (CP) Formulation

Unlike SAT solving, which decomposes problems into propositional clauses, Constraint Programming (CP) preserves high-level integer domain abstractions. Defining an array of integer variables $Q = [Q_0, \dots, Q_{N-1}]$, where $Q_i \in \{0, \dots, N-1\}$ denotes the column of the queen placed on row i (0-based, matching the implementation; one queen per row is implicit in the model), the entire problem is captured using three global constraints:

- $\text{AllDifferent}(Q_0, \dots, Q_{N-1})$ (columns)
- $\text{AllDifferent}(Q_0 - 0, \dots, Q_{N-1} - (N-1))$ (diagonals $i - j$ constant)
- $\text{AllDifferent}(Q_0 + 0, \dots, Q_{N-1} + (N-1))$ (anti-diagonals $i + j$ constant)

The same model is passed to CP-SAT (`AddAllDifferent`) and to CPLEX CP Optimizer (`all_diff`). It has N integer variables and three global constraints instead of hundreds of thousands of clauses. `AllDifferent` admits generalized arc consistency (GAC) via bipartite matching [12], but the propagation level actually applied is a solver choice; we used the default settings of each solver apart from single-threading and did not force GAC. The two CP solvers also differ in architecture: CPLEX CP Optimizer is a classical CP engine, whereas CP-SAT is a lazy-clause-generation solver whose core is itself a CDCL SAT engine that creates Boolean literals for the integer domains on demand. CP-SAT therefore does use auxiliary Booleans internally; what distinguishes it from our SAT pipeline is that the encoding is produced lazily and combined with native propagators, presolve and its own search heuristics.

6 Computational Experiments

6.1 Theoretical Summary of the Five Encodings

Before turning to the empirical results, Table 1 consolidates the structural profile of each encoding: its auxiliary-variable count, clause complexity, and the strength of *unit-propagation* inference. The relevant

Table 1
Theoretical comparison of the five AMO encodings (as implemented). "ACUP" was verified mechanically for every AMO size $m = 2..16$: asserting any x_k lets unit propagation alone derive $\neg x_j$ for all $j \neq k$. "Aux. defined" states whether the auxiliaries are functionally determined by the x variables, i.e. cannot be True when all x are False.

Encoding	Aux. Variables	Clause Complexity	ACUP	Aux. defined
Binomial (Pairwise)	0	$O(n^2)$	yes	–
Sequential Counter	$n-1$	$3n-4$	yes	no
Commander (1-level)	$O(\sqrt{n})$	$O(n\sqrt{n})$	yes	yes
Binary	$\lceil \log_2 n \rceil$	$n\lceil \log_2 n \rceil$	yes	no
Product (1-level)	$O(\sqrt{n})$	$\approx 3n$	yes	no

notion is arc consistency enforced through unit propagation (ACUP): a translation enforces arc consistency if every value that is logically removed by the constraint is also removed by unit propagation alone, without search [4, 5]. For a cardinality translation, ACUP means that the CNF prunes exactly as aggressively as the original constraint.

Table 1 rules out one tempting explanation for runtime differences. All five encodings are correct AMO translations and all five achieve ACUP: with respect to the queen variables, unit propagation prunes exactly as much under Product or Binary as under Binomial. ACUP holds for every size m, by a short argument per encoding: in Binomial each $\neg x_j$ follows from a binary clause; in Sequential x_k forces $s_k \Rightarrow s_{k+1} \Rightarrow \cdots$, which falsifies every later x_j through $(\neg s_{j-1} \vee \neg x_j)$, and forces $\neg s_{k-1} \Rightarrow \neg s_{k-2} \Rightarrow \cdots$, which falsifies every earlier x_j through $(\neg x_j \vee s_j)$; in Binary x_k fixes all bits, so every other x_j has a clause whose bit literal is now false; in Commander x_k sets its own commander, which falsifies the other commanders and hence all variables in other groups, while the pairwise clauses handle its own group; Product was argued in Section 4.3. The mechanical check for $m = 2..16$ confirms these arguments empirically; it does not replace them. Both properties are checked by `src/verify_propagation.py` and by the unit tests, which additionally enumerate all solutions for $N \leq 7$ under every encoding and compare them with brute force. The encodings differ only in size and in how tightly the auxiliaries are tied to the original variables: in Sequential, Binary and Product an auxiliary can be True with no queen placed, while in Commander it cannot. Neither property changes which assignments are pruned by propagation, so any runtime differences must come from search dynamics (decision heuristics, clause learning), which we examine empirically in Section 6.6.

6.2 Experimental Setup

Experiments were conducted across baseline dimensions $N \in \{8, 10, 15, 20, 25, 30\}$, and subsequently extended to two large scales ($N = 40, 50$) and two stress-test scales ($N = 100, 200$) to evaluate scalability frontiers and pinpoint breakdown points. Three distinct families of exact solvers were evaluated: (i) SAT solving with 5 encodings utilizing the Glucose3 CDCL backend; (ii) Constraint Programming with CP-SAT (Google OR-Tools) and CPLEX CP Optimizer (IBM); and (iii) Integer Linear Programming with CPLEX MIP (IBM) and Gurobi. For instances $N \leq 30$, each configuration was executed across 3 repetitions and averaged; for extended benchmarks, timed-out configurations were run once and converging ones three times (except Sequential at $N = 50$, run once). Standard deviation, minimum and maximum of the solve time are recorded in the CSV files alongside the mean. Timing uses the monotonic high-resolution clock `time.perf_counter()`. All solvers were restricted to a single thread (`num_search_workers=1` for CP-SAT, `threads=1` for CPLEX MIP, `Threads=1` for Gurobi, `Workers=1` for CPLEX CP; the PySAT backends are sequential). Solutions across all paradigms—not merely SAT—were independently validated by a shared verification oracle. Model generation time (Python) and solving time were measured separately. For SAT, *Solve* is the time of the `solve()` call only, excluding loading the clauses into the solver; for CP-SAT, CPLEX MIP and Gurobi it is the wall-clock time of the solve call, which includes the solver's internal model translation and presolve; for CPLEX CP it additionally includes starting the external `cpoptimizer` process and transferring the model. A cross-solver evaluation across five backends (Glucose3, Glucose4, CaDiCaL, MapleSAT, CryptoMiniSAT5) and a first-feasible-solution fairness benchmark for CP/MIP (Section 6.7) were also conducted. Measurement noise and input-order sensitivity are analysed in Sections 6.5 and 6.6.

Table 2
Scalability stress-test at $N = 100$ and $N = 200$ (in seconds; average of 3 runs for converging methods, 1 run for timed-out methods). Time limit: 40s for SAT, 60s for CP-SAT. "–" denotes a timeout, or not applicable (clauses for CP-SAT).

Encoding / Solver	N	Clauses	Gen (s)	Solve (s)
Binomial	100	1 646 850	0.575	0.183
CP-SAT	100	–	0.001	0.641
Sequential	100	117 762	0.044	– (timeout)
Commander	100	224 924	0.066	– (timeout)
Binary	100	268 974	0.079	– (timeout)
Product	100	116 570	0.048	– (timeout)
Binomial	200	13 253 700	4.618	– (timeout)
CP-SAT	200	–	0.002	5.897

Environment. All experiments ran on a laptop with an Intel Core i7-13650HX CPU and 24 GB RAM under Windows 11, using Python 3.12, PySAT 1.9 (Glucose 3.0 and the other backends bundled with it), Google OR-Tools 9.15, IBM CPLEX 22.2 Community Edition (`cplex` 22.2.0.1 and `docplex` 2.32 for MIP; CP Optimizer from CPLEX Optimization Studio Community 22.2), and Gurobi 13.0 with the size-limited pip licence. Each SAT solve runs in a separate process so that a timeout can be enforced; in the original runs the timeout clock started before the CNF was transferred to that process, which matters only for the multi-million-clause instances (see Section 6.10). The harness has since been corrected to start the clock after the CNF is loaded. The supplementary experiments (ordering study, CPLEX CP re-run, noise re-measurement) used the same machine and library versions; the $N = 100$ part of the ordering study was run under Python 3.14 instead of 3.12, which does not affect the native solver code.

Table 2 shows where the SAT encodings break down **on the original variable order**. At $N = 100$, Binomial generates **1,646,850 clauses**—over 38 times more than at $N = 30$—yet solves in merely 0.183**s**. In contrast, *all* auxiliary-variable encodings (Sequential, Commander, Binary, Product) **exceed the** 40**s feasibility threshold**, despite producing only $\frac{1}{14}$ to $\frac{1}{6}$ as many clauses. This shows that **CNF size does not dictate solving speed**; whether the auxiliary-variable encodings are intrinsically slow or merely unlucky with the default variable order is examined in Section 6.6. At $N = 200$, Binomial itself breaks down: it takes 4.6s merely to *generate* 13,253,700 clauses and does not finish within the limit. In the original harness roughly half of the 40s budget was spent transferring and loading this CNF into the solver process; a re-run with the corrected harness, whose clock starts only after loading (`results/binomial_n200_long_run.csv`), confirms, however, that Glucose3 does not finish within 300s of pure solving time, so the conclusion stands. CP-SAT, which builds its Boolean representation lazily, solves $N = 100$ in 0.64s and $N = 200$ in 5.9s. Note that the time limits in this table differ between SAT (40s) and CP-SAT (60s); since CP-SAT finished far below either limit, this does not affect the comparison.

6.3 Encoding-Size vs Runtime Analysis

Experimental findings in Figure 1 corroborate theoretical predictions: although each Binomial AMO is quadratic in its length, the whole Binomial CNF grows cubically in N because there are $\Theta(N)$ lines of length up to N, yielding 203,425 clauses at $N = 50$ (compared to 43,255 at $N = 30$). In contrast, Sequential, Binary, Commander, and Product maintain compact formulas ranging between 28,887 and 57,219 clauses at $N = 50$, achieved through auxiliary variable compression. Consequently, the gap in formula size widens substantially with larger board sizes. For Binomial the total can be stated exactly. Rows contribute N ALO clauses and $N\binom{N}{2}$ AMO clauses, columns another $N\binom{N}{2}$, each diagonal direction $\sum_\ell \binom{\ell}{2} = 2\binom{N}{3} + \binom{N}{2}$ over its diagonals of length ℓ, and symmetry breaking $\lfloor N/2 \rfloor$ unit clauses, giving

$$|\mathrm{CNF}_{\mathrm{Binomial}}(N)| = \frac{5N^3 - 6N^2 + 4N}{3} + \left\lfloor \frac{N}{2} \right\rfloor, \tag{13}$$

which reproduces the measured counts exactly (43,255 at $N = 30$, 1,646,850 at $N = 100$, 13,253,700 at $N = 200$).

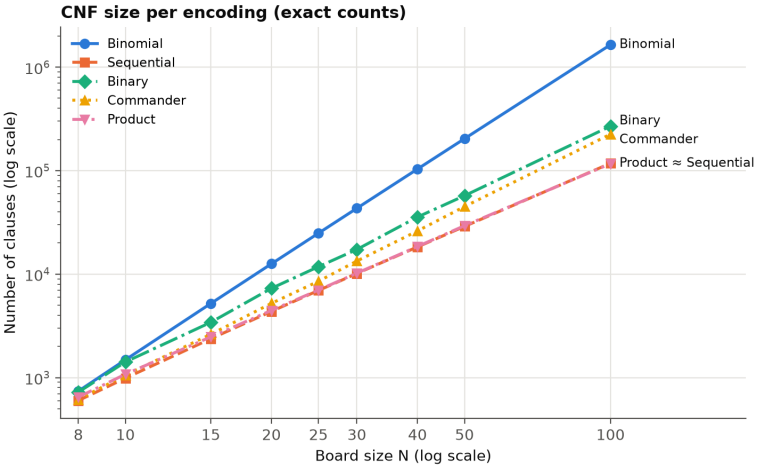


Figure 1: Number of clauses per encoding as a function of N (both axes logarithmic; exact counts computed from the encoder). On log–log axes the slopes reflect the growth order: summing the per-line AMO cost over the $\Theta(N)$ rows, columns and diagonals gives $\Theta(N^3)$ for Binomial, $\Theta(N^{2.5})$ for Commander, $\Theta(N^2 \log N)$ for Binary and $\Theta(N^2)$ for Sequential and Product. Product and Sequential are close at larger sizes: for $N \geq 25$ their clause counts differ by 0.8–1.6% (maximum 1.61% at $N = 50$), so their curves nearly overlap; at $N \leq 20$ the relative difference is 2.2–9.1%.

Table 3
Runtime performance at $N = 30$ (average of 3 repetitions), in seconds.

Encoding / Solver	Clauses	Variables	Gen (s)	Solve (s)
Binomial	43 255	900	0.016 11	0.012 56
Sequential	10 137	4322	0.001 31	0.207 53
Binary	17 301	1666	0.009 27	9.650 24
Commander	13 373	1606	0.005 43	0.248 49
Product	10 215	2426	0.002 42	14.448 40
CP-SAT	–	–	0.001 00	0.077 55
CPLEX-CP	–	–	0.000 61	0.015 70
CPLEX-MIP	–	–	0.016 53	0.011 33
Gurobi-MIP	–	–	0.007 50	0.000 63

Table 3 reports runtime metrics at $N = 30$. Timing is decomposed into Python clause generation (*Gen*) and solver execution (*Solve*).

Table 3 highlights a crucial insight: for Binomial, the majority of runtime (0.016s out of 0.029s, or approximately 56%) is consumed by Python clause generation rather than the SAT solver core. A similar pattern emerges in CPLEX-MIP, where modeling overhead (0.017s) exceeds solving time (0.011s). This underscores the necessity of decoupling *formulation overhead* from *solving duration* when evaluating constraint encodings.

6.4 Runtime Analysis on the Original Order

Figure 2 shows solve times on the original variable order. At $N \leq 15$ all SAT encodings solve within 0.03s; divergence appears from $N = 20$, where Product (0.61s) is already 277× slower than Binomial (0.0022s). At $N = 30$ (Table 3) Product needs 14.45s and Binary 9.65s, against 0.25s for Commander, 0.21s for Sequential and 0.0126s for Binomial (0.029s including generation). A smaller CNF is therefore clearly no guarantee of faster solving: Binomial has the largest CNF and is the fastest encoding at every size up to $N = 100$ on this order.

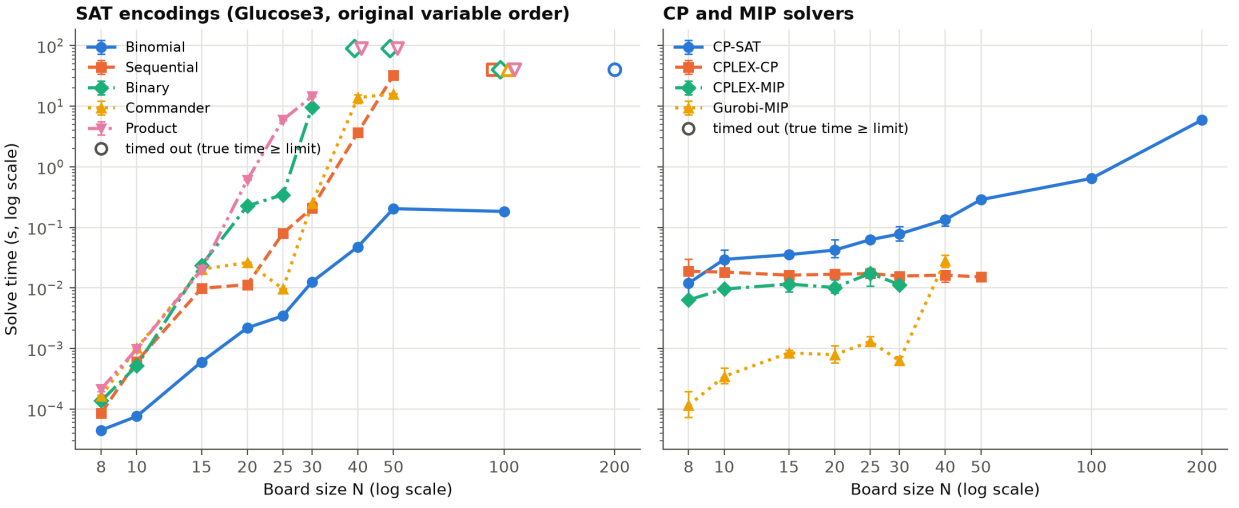


Figure 2: Solve time versus N (both axes logarithmic). Left: the five SAT encodings with Glucose3 on the original variable order; right: CP and MIP solvers. Points are means over repetitions and bars span min–max. Hollow markers denote timeouts, drawn at the time limit of that run, so the true time is at least that value; when several methods time out at the same N their markers are spread horizontally around N. Time limits: 100s ($N \leq 30$), 90s ($N = 40, 50$), 40s for SAT and 60s for CP-SAT ($N = 100, 200$).

6.5 Measurement Noise and Statistical Reliability

The repeated measurements reveal a systematic pattern in their dispersion that must be addressed before the rankings below are interpreted. We use the coefficient of variation (CV, the population standard deviation of the three repetitions divided by their mean). For the SAT encodings the CV at $N = 30$ is 0.4–4.8% and stays below 5% for every repeated configuration with $N \geq 20$ except Commander at $N = 40$ (13.7%, runs between 11.2 and 15.5s). The two CP solvers, whose runtimes are a few to a few tens of milliseconds, are much noisier at small sizes: across $N = 8$–30, CP-SAT shows CVs of 3.9–44.4% and CPLEX-CP of 3.9–43.0%.

We attribute this mainly to fixed overheads rather than to the search itself: when the measured interval is only milliseconds long, one-off costs such as initialisation and operating-system scheduling dominate. A dedicated re-measurement of CP-SAT with 10 repetitions per size (`results/cpsat_noise_per_run.csv`, sample standard deviation) shows the CV decreasing as the measured interval lengthens: 147% at $N = 8$ (mean 5.3ms), 11.4% at $N = 30$ (38ms) and 1.4% at $N = 50$ (116ms). At $N = 8$ the dispersion is caused almost entirely by the first repetition (27.7ms versus a mean of 2.8ms for the other nine), consistent with one-off initialisation; at $N = 30$ the first repetition is only 2% slower than the mean of the rest, and at $N = 50$ it is faster. Once solving takes more than about a hundred milliseconds, the measurement stabilises.

The practical consequence is that differences between millisecond-scale configurations must be checked against the noise before they are interpreted. At $N = 30$ CP-SAT averages 0.0776 ± 0.0188s and CPLEX-CP 0.0157 ± 0.0006s (mean $\pm$ one standard deviation); the gap of 0.062s is 3.3 times the combined standard deviation and even the slowest CPLEX-CP repetition (0.0162s) is faster than the fastest CP-SAT run at $N = 30$ in any session (0.0349s), so CPLEX-CP is faster in wall-clock terms. With only three repetitions, however, no formal significance test is meaningful. Note also that CPLEX-CP's wall-clock time is almost entirely overhead: in a separate run that records the engine-reported time (`results/solver_stats.csv`), the engine needs only 1–4ms for every N from 8 to 50, while the wall-clock time of the same calls is 14–16ms (and 0.25s for the very first call, which starts the external `cpoptimizer` process). The nearly constant 15–19ms across all N in the main tables reflects this overhead.

Within-session dispersion also understates the true uncertainty. CP-SAT at $N = 30$ averaged 0.078s in the main benchmark, 0.038s in the 10-repetition re-measurement and 0.036s in the first-solution benchmark (Table 5): the spread between sessions is larger than any within-session standard deviation. Millisecond-scale timings should therefore be read as order-of-magnitude indications only. For the SAT encodings an even

Table 4
Glucose3 solve time (seconds) over ten equivalent CNFs per configuration: the original order (seed 0) and nine random variable/clause permutations. "≥ 60" denotes a timeout; the median treats timeouts as ≥ 60s and is therefore exact whenever fewer than half of the runs time out.

Encoding	N	Original	Median	Min	Max	Timeouts
Binomial	30	0.012	0.002	0.001	0.012	0/10
Sequential	30	0.201	0.153	0.012	2.253	0/10
Binary	30	6.412	0.012	0.003	6.412	0/10
Commander	30	0.243	0.008	0.003	0.243	0/10
Product	30	13.271	0.012	0.008	13.271	0/10
Binomial	50	0.194	0.005	0.003	0.194	0/10
Sequential	50	23.906	12.592	0.023	≥ 60	3/10
Binary	50	≥ 60	0.024	0.004	≥ 60	1/10
Commander	50	12.776	0.016	0.009	12.776	0/10
Product	50	≥ 60	1.364	0.010	≥ 60	2/10
Binomial	100	0.185	0.027	0.023	0.185	0/10
Sequential	100	≥ 60	≥ 60	≥ 60	≥ 60	10/10
Binary	100	≥ 60	≥ 60	0.412	≥ 60	9/10
Commander	100	≥ 60	0.025	0.012	≥ 60	1/10
Product	100	≥ 60	≥ 60	0.038	≥ 60	8/10

larger source of variation exists that repetition cannot reveal at all, namely sensitivity to input order (Section 6.6).

6.6 Sensitivity to Variable and Clause Order

A CDCL solver is deterministic: given the same CNF in the same order it performs the same search, so the repetitions above can only measure timer noise. Its search does, however, depend on the order of variables and clauses, which affects initial decision order and tie-breaking. To separate the effect of an encoding from the effect of one particular order, every encoding was solved at $N \in \{30, 50, 100\}$ on ten logically equivalent CNFs: seed 0 is the original CNF, and seeds 1–9 rename the variables by a random permutation and shuffle the clauses and the literals within each clause (`src/robustness.py`, Glucose3, 60s limit per run, one run per seed). All 116 runs that finished returned solutions accepted by the verification oracle. Table 4 summarises the results and Figure 3 shows every run.

Four observations follow.

(1) The original order is unfavourable for every encoding. In 13 of the 15 configurations the original order is the slowest of the ten runs or times out (the exceptions are Sequential at $N = 30$, where one permuted run took 2.25s, and at $N = 50$, where three permuted runs timed out); even Binomial is 5.8×, 41.7× and 6.8× slower on its original order than its median at $N = 30, 50, 100$. The row-major numbering of the queen variables, with auxiliaries appended at the end, is therefore not a neutral choice.

(2) At $N = 30$ most of the apparent gaps are artefacts of that order. Product drops from 13.3s to a median of 0.012s (1083×), Binary from 6.4s to 0.012s (515×) and Commander from 0.24s to 0.008s (32×). Their medians are at most 6.1× Binomial's (0.002s), whereas the original-order results of Table 3 suggested gaps of up to three orders of magnitude.

(3) At larger N genuine differences remain. Sequential is the only encoding that is already slow at $N = 30$ on most orders (median 74× Binomial's); at $N = 50$ its median is 12.6s with 3/10 timeouts, and at $N = 100$ all ten runs time out. Binary and Product survive most permutations at $N = 50$ (medians 0.024s and 1.36s) but fail on 9/10 and 8/10 orders at $N = 100$. Their solved runs at $N = 100$ (0.41s and 0.04–0.28s) show that the instance is easy for these encodings in principle, but the solver rarely reaches such a run.

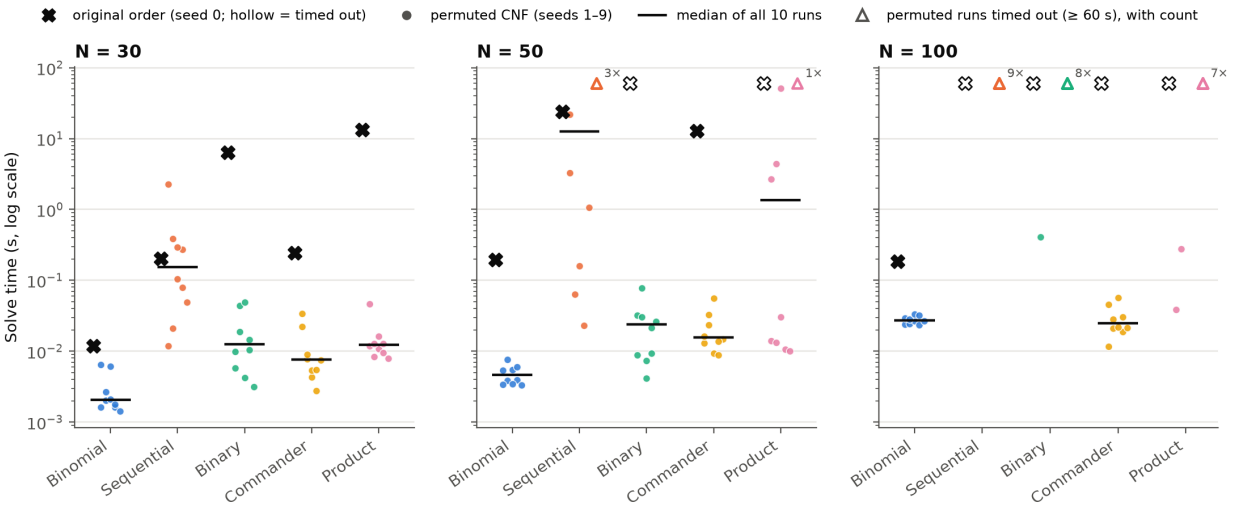


Figure 3: Every run of the ordering study (logarithmic time axis). Crosses mark the original order (hollow when it timed out); dots are the nine permuted CNFs; the bar is the median of all ten runs (omitted when at least half time out); triangles at the 60s limit mark permuted runs that timed out, with their count.

(4) Binomial and Commander are robust. Both solve every permuted CNF at every size, with medians of 0.027s and 0.025s at $N = 100$; the earlier conclusion that "every auxiliary-variable encoding fails at $N = 100$" (Table 2) is true only for the original order and does not hold for Commander.

Note finally that Commander is the only auxiliary encoding whose auxiliaries are functionally defined (Table 1), while the three encodings that degrade at $N = 100$ all have auxiliaries that can be True with no queen placed. This correlation is consistent with the hypothesis that unconstrained auxiliaries give the solver many irrelevant decision points, but with five encodings it is an observation, not a demonstration.

Interpreting the SAT Results

What the data support. (i) CNF size does not predict solve time: Binomial, the largest encoding, never times out in the ordering study and has the lowest median time at $N = 30$ and 50; at $N = 100$ only Commander matches it (0.025s vs. 0.027s). (ii) Propagation strength does not explain the differences, since all five encodings are ACUP (Table 1). (iii) Single-order measurements exaggerate the gaps: at $N = 30$ the slowdowns of Binary, Commander and Product shrink by factors of 515, 32 and 1083 once the order is randomised (Table 4). (iv) Genuine differences appear at larger N: Sequential degrades consistently from $N = 30$ on, and Binary and Product fail on most orders at $N = 100$, while Commander stays as fast as Binomial.

What remains a hypothesis. Two mechanisms are plausible but were not measured. First, auxiliaries that are not functionally defined (Sequential, Binary, Product) give the decision heuristic variables whose values carry no information about the board; Commander, whose auxiliaries are defined, is the only auxiliary encoding that stays robust. Second, Sequential's chain $s_1 \Rightarrow s_2 \Rightarrow \cdots \Rightarrow s_{n-1}$ makes conflicts span many decision levels, which may produce learned clauses with high Literal Block Distance (LBD) that Glucose deletes early [10]. Testing either explanation would require solver statistics (decisions on auxiliary variables, LBD distributions), which our harness does not record.

6.7 Fair SAT vs MIP Benchmark: First Feasible Solution

SAT solvers stop at the **first feasible solution**, whereas MIP solvers are built to prove optimality, so a difference in *termination criterion* could bias the comparison. To test this, we added a **first-solution** mode for CP/MIP solvers: Gurobi sets `SolutionLimit=1`, CPLEX MIP sets `mip.limits.solutions=1`, and CP-SAT sets `stop_after_first_solution=True`. Table 5 compares both modes on identical benchmark runs (averaged across 5 repetitions).

Surprising finding: the first-solution mode causes virtually no difference ($\Delta \approx 1.0$ across all tested instances). Two key methodological factors explain this phenomenon. First, N-Queens is

Table 5
Comparison between default mode and first-feasible-solution mode across board sizes N, in seconds (average of 5 runs). "Δ" represents the ratio $\text{Solve}_{\text{default}}/\text{Solve}_{\text{first-sol}}$; $\Delta \approx 1$ indicates that the stopping criterion causes no performance discrepancy.

Solver	N	Solve – default (s)	Solve – first-sol (s)	Δ
Gurobi-MIP	20	0.000 237	0.000 268	0.88
CPLEX-MIP	20	0.003 565	0.003 683	0.97
CP-SAT	20	0.015 454	0.016 063	0.96
Gurobi-MIP	30	0.000 343	0.000 335	1.02
CPLEX-MIP	30	0.005 809	0.005 784	1.00
CP-SAT	30	0.036 274	0.036 926	0.98
Gurobi-MIP	40	0.014 091	0.013 585	1.04
CP-SAT	40	0.068 825	0.069 390	0.99

fundamentally a *feasibility* problem rather than an optimization problem: the MIP formulations constructed here **contain no objective function**. Consequently, MIP solvers terminate immediately upon finding the first feasible integer solution—imposing `SolutionLimit=1` yields no acceleration. (Independent verification in `results/solver_stats.csv`: for every N from 8 to 30 Gurobi reports `SolCount=1`, `ObjVal=ObjBound=0` and `NodeCount=0`; at $N = 40$ it explores a single node.) Second, CP-SAT has no objective either, so `stop_after_first_solution` changes nothing. *Thus, any conjecture that MIP is disadvantaged due to optimality proofs is refuted for this problem.*

The node counts also tell us *why* Gurobi is fast: up to $N = 30$ it does not branch at all, and at $N = 40$ it needs a single node. The solution is found during presolve and root-node primal heuristics, so its advantage cannot be attributed to "branching on primary variables"; it reflects the strength of these generic heuristics on a highly structured assignment-type model. Under the first-solution criterion Gurobi (0.000335s at $N = 30$) is about 37× faster than the fastest Glucose3 configuration (Binomial, 0.0126s).

Overall, the fastest method depends on N. In the default runs Binomial with Glucose3 is fastest at $N \leq 15$ (e.g. 0.045ms vs. 0.115ms for Gurobi at $N = 8$), **Gurobi-MIP** at $N = 20$–30 (0.00063s at $N = 30$, about 20× faster than Binomial), CPLEX-CP at $N = 40$ and 50 (0.016s and 0.015s), and Binomial again at $N = 100$ (0.18s vs. 0.64s for CP-SAT), where the MIP solvers could not be run. At $N = 30$ **CPLEX-MIP** follows Gurobi with 0.0113s. In the CP domain, **CPLEX-CP** (0.0157s) and **CP-SAT** (0.0776s) both lie *behind* Binomial with Glucose3 (0.0126s). The paradigm ranking is thus not "CP/MIP above SAT": a well-chosen SAT encoding is competitive with both CP solvers.

6.8 Cross-Solver SAT Benchmark

To disentangle solver architecture from encoding formulation, we evaluated identical CNF models across five distinct PySAT backends: Glucose3, Glucose4, CaDiCaL, MapleSAT, and CryptoMiniSAT5. Table 6 summarizes the results at $N = 30$ for Binomial and Sequential encodings. All solvers produced valid solutions. The CNF given to every solver is identical; the differences in the *Gen* column are timing noise of the Python generator, not solver effects. The backends are the PySAT builds of Glucose 3.0, Glucose 4.1, CaDiCaL 1.9.5 and MapleSAT, and CryptoMiniSat 5.11 through its `pycryptosat` binding.

The ranking of the solvers **partially reorders between encodings**. CryptoMiniSAT5 is fastest under both (0.00105s on Binomial, 0.04713s on Sequential), but the others swap places: CaDiCaL is last on Binomial and second on Sequential, while Glucose4, tied with Glucose3 for second place on Binomial (0.0126s), is last on Sequential. The efficiency of a (solver, encoding) pair therefore cannot be predicted from the encoding alone. Switching from Glucose3 to CryptoMiniSAT5 yields a 4.4× speedup on Sequential and a 12× speedup on Binomial. We do not attribute this to a specific feature of CryptoMiniSAT5: the instances contain no XOR constraints, and each cell of the table is a single run on a single variable order, so given the ordering sensitivity shown in Section 6.6 these results are indicative only.

Notably, CryptoMiniSAT5 with Binomial (0.00105s) is faster at $N = 30$ than CP-SAT, CPLEX-CP and CPLEX-MIP, and within a factor of two of Gurobi-MIP (0.00063s). Although these numbers come from

Table 6
Cross-solver SAT comparison at $N = 30$ (single run), in seconds. Gen denotes the shared clause generation time.

Solver	Encoding	Clauses	Gen (s)	Solve (s)
CryptoMiniSAT5	Binomial	43 255	0.005 498	0.001 052
Glucose3	Binomial	43 255	0.033 204	0.012 604
Glucose4	Binomial	43 255	0.005 174	0.012 626
MapleSAT	Binomial	43 255	0.004 159	0.023 773
CaDiCaL	Binomial	43 255	0.009 116	0.046 083
CryptoMiniSAT5	Sequential	10 137	0.001 347	0.047 134
CaDiCaL	Sequential	10 137	0.001 168	0.129 670
Glucose3	Sequential	10 137	0.001 867	0.207 402
MapleSAT	Sequential	10 137	0.001 230	0.543 011
Glucose4	Sequential	10 137	0.001 166	0.893 541

Table 7
Scalability benchmark at $N = 40$ (average of 3 runs for converging methods, 1 run for timed-out methods; 90s time limit), in seconds. "–" denotes timeout or model/license limit exceeded.

Encoding / Solver	Clauses	Variables	Gen (s)	Solve (s)
Binomial	103 540	1600	0.056 37	0.047 78
Sequential	18 312	7762	0.004 99	3.683 46
Binary	35 604	2776	0.007 87	– (timeout)
Commander	26 116	2724	0.006 25	13.822 34
Product	18 484	3998	0.004 08	– (timeout)
CP-SAT	–	–	0.001 13	0.133 59
CPLEX-CP	–	–	0.000 76	0.016 28
CPLEX-MIP	–	–	–	– (exceeded 1000-var limit)
Gurobi-MIP	–	–	0.016 53	0.028 07

different sessions and should be compared at the order-of-magnitude level only (Section 6.5), they do not support the claim that Boolean reduction is inherently slower than CP or MIP on this problem; what matters is the combination of encoding and solver.

6.9 Scalability Verification at $N = 40$ and $N = 50$

To verify that these dynamics extend beyond small-to-moderate scales, we evaluated $N = 40$ under a 90-second timeout. Results (Table 7) show that **Binomial remains the fastest SAT encoding** (0.048s solve time, 3-run average), consolidating—rather than reversing—its lead from $N = 30$: it is now 77× faster than Sequential (3.68s) and 289× faster than Commander (13.82s), while **Binary and Product exceed the 90s feasibility threshold**. Generating Binomial's 103,540 clauses costs only 0.056s, so its larger CNF is not a practical obstacle at this size.

Among the other paradigms, **CPLEX-CP** needs 0.016s, **Gurobi-MIP** 0.028s (one branch-and-bound node) and **CP-SAT** 0.134s. An important tooling observation: **CPLEX-MIP Community Edition** reached its 1,000-variable ceiling at $N = 40$ ($1,600$ variables) and could not execute. Because the original-order SAT results at this size are strongly affected by input order (Section 6.6), the table should be read as one sample per encoding rather than as a definitive ranking.

At the harsher scale of $N = 50$, this trajectory is fully reaffirmed (Table 8). **Binomial** remains the fastest SAT encoding (0.203s solve time for $203,425$ clauses), whereas **Binary** and **Product** fail completely within the 90-second limit. Commander (15.9s) and Sequential (32.2s, single run) still finish but are two orders of magnitude slower than Binomial. Section 6.6 shows how much of this gap is due to the encodings themselves and how much to the particular input order.

Table 8
Scalability benchmark at $N = 50$ (average of 3 runs for converging methods except Sequential, which was run once; 1 run for timed-out methods; 90s time limit), in seconds. "–" denotes timeout or model/license limit exceeded.

Encoding / Solver	Clauses	Variables	Gen (s)	Solve (s)
Binomial	203 425	2500	0.083 85	0.203 34
Sequential	28 887	12 202	0.005 12	32.205 65
Binary	57 219	4036	0.041 51	– (timeout)
Commander	44 905	4114	0.010 87	15.933 17
Product	29 353	5910	0.005 48	– (timeout)
CP-SAT	–	–	0.002 33	0.286 26
CPLEX-CP	–	–	0.000 53	0.015 17
CPLEX-MIP	–	–	–	– (exceeded 1000-var limit)
Gurobi-MIP	–	–	–	– (exceeded license limit)

6.10 Threats to Validity

- **Single input order.** Tables 2, 3, 6, 7 and 8 and Figure 2 use one fixed variable and clause order per instance. Section 6.6 shows that this order alone can change Glucose3 runtimes by two to three orders of magnitude, so per-cell comparisons in those tables should not be over-interpreted.
- **One instance per size.** N-Queens has a single satisfiable instance per N; results may not transfer to other problems built from AMO constraints.
- **Different time limits.** The limits differ between experiments (40, 60, 90, 100 and 300s). Timed-out entries are reported as such and never averaged with finished runs.
- **Timeout accounting.** In the original runs the SAT timeout included transferring the CNF to the solver process. This is negligible below 10^6 clauses but consumed about half of the budget for Binomial at $N = 200$ (Table 2); the harness now starts the clock after loading.
- **Re-measured CPLEX-CP.** The original CPLEX-CP runs did not set `Workers=1`, so CP Optimizer used all CPU cores. All CPLEX-CP figures in this report come from a single-threaded re-run; the original multi-threaded numbers are kept in `results/cplexcp_multithread_original.csv`.
- **Asymmetric symmetry breaking.** Only the SAT encodings include the first-row symmetry-breaking clauses.
- **Licence limits.** CPLEX Community Edition and the pip Gurobi licence cap model size, so MIP results are missing at $N \geq 40$ (CPLEX) and $N \geq 50$ (Gurobi).
- **Timing noise.** Millisecond-scale timings vary between sessions by more than 2× (Section 6.5). Even long deterministic runs vary: the identical original-order CNF of Binary at $N = 30$ took 9.65s in the main benchmark and 6.41s in the ordering study, presumably owing to CPU frequency and thermal state of the laptop. This is small compared with the ordering effects but rules out precise cross-session ratios; in particular, absolute times in Table 4 and in Tables 3–8 come from different sessions and should not be compared digit by digit.

7 Conclusion

This study evaluated five SAT encodings of the AMO constraint on the N-Queens problem, together with CP and MIP solvers, and checked its own measurements for two common pitfalls. Its findings are as follows.

- **All five encodings are correct and ACUP.** Differences in solving time therefore cannot be attributed to weaker unit propagation; they arise from search dynamics.
- **CNF size does not predict speed.** Binomial produces the largest CNF ($\frac{5N^3-6N^2+4N}{3} + \lfloor N/2 \rfloor$ clauses) yet is the fastest SAT encoding on the original order up to $N = 100$ and never times out on any of the 10 orders tested. Its limit is size itself: at $N = 200$ the 13.25 million clauses take 4.6s to generate and Glucose3 does not finish within 300s of solving.

- **Single-order results are unreliable.** The original variable order was the slowest of ten equivalent orders (or timed out) in 13 of 15 configurations. At $N = 30$ the apparent gaps between encodings shrink from up to three orders of magnitude to at most 6.1× Binomial's median once the order is randomised (Sequential excepted).
- **Genuine differences appear at scale.** Sequential degrades consistently from $N = 30$ on and fails on every order at $N = 100$; Binary and Product fail on 9/10 and 8/10 orders at $N = 100$; Commander remains as fast as Binomial (0.025s vs. 0.027s median). The three degrading encodings are exactly those whose auxiliaries are not functionally defined, a correlation that merits further study with solver statistics.
- **Paradigms.** Gurobi solves every size its licence admits (up to $N = 40$) in at most 0.04s, with zero branch-and-bound nodes up to $N = 30$ and one at $N = 40$; CPLEX CP needs 15–19ms of wall-clock time but only 1–4ms inside the engine; CP-SAT scales to $N = 200$ in 5.9s. A good (encoding, solver) pair is competitive: CryptoMiniSAT5 with Binomial solves $N = 30$ in 1.05ms, faster than both CP solvers and CPLEX MIP. Stopping criteria do not explain the differences, since the MIP models have no objective.
- **Methodology.** Repeating a deterministic solver on one input measures only timer noise; between-session variation exceeds within-session variation for millisecond-scale timings; and the CPLEX CP configuration had to be re-measured after a threading error was found. Future work should report distributions over input permutations and record solver statistics to test the mechanisms proposed in Section 6.6.

Data Availability

Reproducibility Statement

Source code, raw CSV results and unit tests are available at `https://github.com/Messionly1/NQueens-SAT-Solving`. Table 1 is reproduced by `src/verify_propagation.py`, Section 6.6 by `src/robustness.py`, the noise re-measurement by `src/noise_check.py`, the Gurobi node counts and CPLEX CP engine times by `src/solver_stats.py`, and all figures by `src/plot_charts.py` from the CSV files in `results/`; the README gives the full commands.

References

[1] Bell, J., Stevens, B., 2009. A survey of known results and research areas for n-queens. Discrete Mathematics 309, 1–31.
[2] Warners, J.P., 1998. A linear-time transformation of linear inequalities into conjunctive normal form. Information Processing Letters 68, 63–69.
[3] Sinz, C., 2005. Towards an optimal CNF encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming (CP 2005). Springer, pp. 827–831.
[4] Gent, I.P., 2002. Arc consistency in SAT, in: Proceedings of the 15th European Conference on Artificial Intelligence (ECAI 2002). IOS Press, pp. 121–125.
[5] Bailleux, O., Boufkhad, Y., 2003. Efficient CNF encoding of Boolean cardinality constraints, in: Principles and Practice of Constraint Programming (CP 2003). Springer, LNCS 2833, pp. 108–122.
[6] Klieber, W., Kwon, G., 2007. Efficient CNF encoding for selecting 1 from N objects, in: Fourth Workshop on Constraints in Formal Verification (CFV).
[7] Chen, J., 2010. A new SAT encoding of the At-Most-One constraint, in: Tenth International Workshop on Constraint Modelling and Reformulation (ModRef).
[8] Gomes, C.P., Selman, B., Crato, N., Kautz, H., 2000. Heavy-tailed phenomena in satisfiability and constraint satisfaction problems. Journal of Automated Reasoning 24, 67–100.
[9] Marques-Silva, J., Lynce, I., Malik, S., 2009. Conflict-driven clause learning SAT solvers, in: Handbook of Satisfiability. IOS Press, pp. 131–153.
[10] Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern SAT solvers, in: International Joint Conference on Artificial Intelligence (IJCAI). pp. 399–404.
[11] Rossi, F., van Beek, P., Walsh, T. (Eds.), 2006. Handbook of Constraint Programming. Elsevier.
[12] Régin, J.C., 1994. A filtering algorithm for constraints of difference in CSPs, in: AAAI Conference on Artificial Intelligence. pp. 362–367.
[13] Bertsimas, D., Tsitsiklis, J.N., 1997. Introduction to Linear Optimization. Athena Scientific, Belmont, MA.